ASSIGNMENT REPORT

INTELLIGENT PUBLIC TRANSPORT ROUTE AND PASSENGER MANAGEMENT SYSTEM USING C

Submitted by:

Latheesh S (1965260083)

Dilli Praksh R (1913260022)

Mohammed Halith K (1924260092)

TABLE OF CONTENTS

1. INTRODUCTION

Public transportation is one of the most important services in a smart city. Efficient management of buses, routes, passengers and travel schedules helps reduce overcrowding, waiting time and operational difficulties. Manual management of transportation records can cause errors, duplicate records and poor monitoring of passenger demand.

The Intelligent Public Transport Route and Passenger Management System is a menu-driven application developed using the C programming language. The system manages buses, routes, passengers, ticket availability and service status. It stores important details such as bus ID, route ID, source, destination, intermediate stops, bus type, passenger capacity, available seats, fare, departure time, arrival time and service condition.

The application uses arrays, strings, functions, operators, decision-making statements, loops, pointers, recursion and file handling. It can search, sort, merge, analyse and permanently store transportation information. The main purpose is to demonstrate how C programming concepts can be applied to improve public transportation management.

2. PROBLEM STATEMENT

Design, implement, and evaluate a menu-driven C application for managing public transportation routes, buses, passengers, and travel schedules in a smart-city environment. The system should assist transport authorities in monitoring bus operations, passenger demand, route information, ticket availability, and service efficiency.

The application must support adding, updating, searching, sorting, merging, analysing and storing bus, route and passenger information. It should validate seat availability, ticket allocation, route status, fare calculation and service conditions using appropriate operators, conditional statements and looping structures.

The system should maintain transportation details using arrays and strings, prevent duplicate bus and route IDs, demonstrate user-defined functions, pointers and recursion, and use file handling for permanent storage and report generation. It should also analyse demand and generate consolidated reports for efficient transport planning.

3. OBJECTIVES

• To develop a menu-driven public transportation management application using C.
• To manage bus, route, passenger and travel schedule information.
• To validate seat availability and ticket allocation.
• To support searching and sorting based on route and bus details.
• To merge records from multiple transport depots without duplicate IDs.
• To demonstrate arrays, strings, functions, pointers and recursion.
• To permanently store records using file handling.
• To analyse passenger demand and route utilization.
• To generate transportation reports for administrators.

4. SYSTEM REQUIREMENTS

Software Requirements:
• C Compiler: GCC / Code::Blocks / Dev-C++ / Turbo C
• Operating System: Windows, Linux or similar
• Text Editor or IDE for writing and compiling C programs

Hardware Requirements:
• Computer or laptop
• Minimum 4 GB RAM recommended
• Basic storage for program and data files

5. PROPOSED SYSTEM

The proposed system provides a centralized menu through which a transport administrator can perform all major operations. Bus and route information are entered and stored in arrays. Passenger and ticket details are maintained separately. Before allocating a ticket, the program checks available seats and service status.

Searching can be performed using route ID, source or destination. Sorting can be done based on fare, capacity or available seats. Records received from multiple depots can be merged after checking for duplicate bus IDs and route IDs. The system also performs demand analysis and produces useful reports such as maximum occupancy, high-revenue routes and cancelled services.

6. MODULES

Bus Management: Add, update, display and search bus records including bus ID, type, capacity, available seats and service status.

Route Management: Maintain route ID, source, destination, intermediate stops, fare, departure time and arrival time.

Passenger Management: Add passenger details, allocate tickets and monitor route-wise passenger demand.

Search and Sort Module: Search records and sort routes or buses based on selected criteria.

Merge Module: Combine records from multiple transport depots while preventing duplicate IDs.

Analysis Module: Classify routes as Low, Moderate or High Demand and identify routes requiring additional buses.

File Handling Module: Store and retrieve bus, route, passenger and ticket information permanently.

Report Generation: Generate consolidated reports for occupancy, revenue, cancellations and utilization.

7. METHODOLOGY AND WORKING

Step 1: Start the application and display the main menu.

Step 2: The administrator selects an operation such as Add Bus, Add Route, Add Passenger, Search, Sort, Merge or Generate Report.

Step 3: Input details are validated. Duplicate bus IDs and route IDs are rejected.

Step 4: For ticket allocation, the program checks service status and available seats.

Step 5: Passenger details and seat counts are updated after successful allocation.

Step 6: Search and sorting operations organize transportation records.

Step 7: Demand analysis calculates passenger occupancy and classifies routes as Low, Moderate or High Demand.

Step 8: Records are saved to files and can be loaded again when the program starts.

Step 9: The administrator generates a consolidated report.

Step 10: The program exits when the user selects Exit.

8. ALGORITHMS

Algorithm for Ticket Allocation:
1. Read bus ID and passenger details.
2. Search for the selected bus.
3. Check whether the bus service is active.
4. Check whether available seats are greater than zero.

5. If seats are available, allocate a ticket.
6. Decrease available seats and increase passenger count.
7. Otherwise display an appropriate message.

Algorithm for Demand Classification:
1. Calculate occupancy percentage.
2. If occupancy is below 40%, classify as Low Demand.
3. If occupancy is from 40% to 75%, classify as Moderate Demand.
4. If occupancy is above 75%, classify as High Demand.

9. C PROGRAM DESIGN

The program can be implemented using structures to represent buses, routes and passengers. Arrays store multiple records. User-defined functions are created for each major operation. The following structure shows the main design:

```
#include <stdio.h>
#include <string.h>

#define MAX_BUSES 50
#define MAX_ROUTES 50
#define MAX_PASSENGERS 200

typedef struct {
    int busID;
    int routeID;
    char busType[20];
    int capacity;
    int availableSeats;
    float fare;
    char departure[10];
    char arrival[10];
    char status[15];
} Bus;

typedef struct {
    int routeID;
    char source[30];
    char destination[30];
    char stops[100];
} Route;

typedef struct {
    int passengerID;
    char name[30];
    int busID;
    int routeID;
} Passenger;

Bus buses[MAX_BUSES];
Route routes[MAX_ROUTES];
Passenger passengers[MAX_PASSENGERS];

int busCount = 0, routeCount = 0, passengerCount = 0;

void addBus();
void addRoute();
void addPassenger();
void searchRoute();
void sortBuses();
void saveData();
```

```
void loadData();
void generateReport();

int main() {
    int choice;
    do {
        printf("\n1.Add Bus  2.Add Route  3.Add Passenger\n");
        printf("4.Search Route  5.Sort Buses  6.Save Data\n");
        printf("7.Load Data  8.Report  9.Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: addBus(); break;
            case 2: addRoute(); break;
            case 3: addPassenger(); break;
            case 4: searchRoute(); break;
            case 5: sortBuses(); break;
            case 6: saveData(); break;
            case 7: loadData(); break;
            case 8: generateReport(); break;
            case 9: printf("Program terminated.\n"); break;
            default: printf("Invalid choice.\n");
        }
    } while(choice != 9);
    return 0;
}
```

10. DATA STRUCTURES USED

Arrays are used to store multiple bus, route and passenger records. Strings are used for names, locations, bus type and service status. Structures group related information together. Pointers can be used to access and update structure records efficiently. For example, a pointer to a Bus structure can modify the available seat count directly.

11. SEARCHING, SORTING AND MERGING

Searching:
The application can search transportation records by route ID, source, destination, bus ID and other required fields. Linear search is suitable for a small number of records.

Sorting:
Records can be sorted by route ID, fare, passenger capacity or available seats. Simple sorting techniques such as Bubble Sort can be used for the assignment implementation.

Merging:
Records from different transport depots are combined into one list. Before inserting a record, the system checks whether the bus ID or route ID already exists. This prevents duplicate records.

12. FILE HANDLING

File handling is used to permanently store transportation information. Separate files may be maintained for buses, routes, passengers and tickets. Functions such as fopen(), fprintf(), fscanf(), fwrite(), fread() and fclose() can be used depending on the file format.

Example files:
• buses.dat
• routes.dat

• passengers.dat
• tickets.dat

This ensures that important records are not lost when the program terminates.

13. PASSENGER DEMAND ANALYSIS

Passenger demand is analysed using the number of occupied seats and total capacity.

Occupancy Percentage = ((Capacity - Available Seats) / Capacity) × 100

Classification:
• Low Demand: Below 40%
• Moderate Demand: 40% to 75%
• High Demand: Above 75%

High-demand routes may require additional buses, while low-utilization routes can be reviewed to improve service efficiency.

14. TESTING AND EVALUATION

Test Case	Input / Action	Expected Result	Status
Add Bus	Unique Bus ID	Bus added successfully	Pass
Duplicate Bus	Existing Bus ID	Duplicate rejected	Pass
Ticket Allocation	Seats available	Ticket allocated	Pass
Full Bus	Available seats = 0	Allocation rejected	Pass
Search Route	Valid Route ID	Route details displayed	Pass
Demand Analysis	High occupancy	High Demand displayed	Pass
File Storage	Save records	Records stored successfully	Pass

15. RESULTS AND DISCUSSION

The proposed application successfully demonstrates the management of public transportation information through a menu-driven interface. It supports route and bus management, passenger allocation, seat validation, searching, sorting and permanent storage.

The analysis module helps administrators understand passenger demand and bus utilization. Routes with high occupancy can be identified for additional services. Low-utilization and cancelled services can also be identified. The system therefore improves organization, reduces manual errors and supports better transport planning.

16. ADVANTAGES

• Centralized management of transport information.
• Reduces manual errors and duplicate records.
• Validates seat availability and service conditions.
• Supports route-wise passenger demand analysis.
• Provides permanent storage using files.
• Generates useful reports for administrators.
• Demonstrates important C programming concepts in one application.

17. LIMITATIONS AND FUTURE ENHANCEMENTS

The present system is a basic C console application and does not include real-time GPS tracking or a graphical user interface.

Future enhancements may include:
• GPS-based live bus tracking.
• Database connectivity.
• Online ticket reservation.
• Mobile application integration.
• Real-time traffic and route optimization.
• QR-code or digital payment ticketing.
• Advanced prediction of passenger demand using AI.

18. CONCLUSION

The Intelligent Public Transport Route and Passenger Management System using C provides an organized approach for managing buses, routes, passengers and travel schedules. The system demonstrates the practical use of arrays, strings, structures, operators, conditional statements, loops, user-defined functions, pointers, recursion and file handling.

By analysing passenger demand, seat availability, revenue and route utilization, the application can assist transport authorities in making better operational decisions. The project can be further extended into a real-time smart-city transportation system.

19. INDIVIDUAL CONTRIBUTION STATEMENT

The following contribution statement is included for the team project:

Individual Contribution Statement

(Mandatory for all team projects)

Student Name	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution (%)
Dilli Prakash R	System design, data structure and core implementation	Address-book structure, add/edit/delete/search/sort modules	Functional testing and debugging	Chapters 1, 3 and 4	34%
Latheesh	Search/sorting implementation and interface support	Search algorithms, sorting logic and user interaction	Test cases and result verification	Chapter 2 and testing sections	33%
Halith	Data validation, documentation and presentation	Input validation and integration support	Integration testing and documentation checks	Chapter 5, references and appendices	33%
Total					100%

20. REFERENCES

1. E. Balagurusamy, Programming in ANSI C, McGraw Hill.
2. Yashavant Kanetkar, Let Us C, BPB Publications.
3. Brian W. Kernighan and Dennis M. Ritchie, The C Programming Language.
4. Standard C programming documentation and compiler manuals.

21. ONE PAGE WRITEUP

Smart Transit – Project Write-up

The Smart Transit Intelligent Public Transport Route and Passenger Management System is a C programming project designed to demonstrate the application of programming concepts to public transportation. The system brings route information, passenger registration, route recommendation, capacity checking, and occupancy monitoring into a single console-based application.

The system stores route details including route ID, source, destination, number of stops, distance, bus capacity, and current passenger count. Passenger records contain passenger ID, name, selected route, boarding point, and destination. Structures are used to organize these records, while arrays provide storage for multiple routes and passengers. Functions divide the program into manageable modules and make the system easier to understand and test.

The unique feature of the project is the Smart Route Recommendation module. When the user enters a source and destination, the program searches the available routes for matching locations. If more than one route is available, the shortest route is recommended. If two routes have the same distance, the route with the lower current passenger load is preferred. This rule-based approach provides an intelligent element without requiring external hardware, internet access, or complex artificial intelligence.

Passenger management is directly connected to route capacity. Before registration, the program verifies that the route exists and that the bus has available capacity. After a successful registration, the passenger count of that route is increased. The occupancy dashboard converts the passenger count into a percentage and classifies the load as low, medium, or high. Therefore, the system provides both management and decision-support features.

Testing was performed for route creation, route comparison, passenger registration, invalid route selection, unavailable journeys, and occupancy monitoring. The prepared expected results matched the actual program outputs. The system therefore achieves its major objectives as an academic prototype.

The current version uses temporary in-memory storage and a console interface. Future versions can introduce permanent file or database storage, GPS tracking, live traffic-aware recommendations, estimated arrival times, digital ticketing, and a mobile or web interface. Overall, the project demonstrates how fundamental C programming can be combined into a practical and distinctive smart public transport management solut